

Build Improvement Plan

system-monitor · Next.js 16.2.12 · Bun 1.3.5 · prepared 25/09/2026 by Kai · every figure below was measured on the current tree (commit d81b50e)

Where the build stands. The toolchain is in better shape than most projects this size: strict TypeScript, zero-warning lint, 93 passing unit tests, a boot-and-render smoke test, and a 40-check QA gate. All of that is green under native arm64 node.

What is wrong. Three of the gates can report green while the thing they guard is broken. The CI dependency-audit step swallows failures, and today that hides two *critical* remote-code-execution advisories in the installed Next.js. The smoke and QA gates exit with an unexplained "code 1" whenever a dev server is already open, and the QA gate passes when the registry is unreachable. The test suite exercises only 9 library files; no route handler, server action, hook, or component is ever imported by a test.

Recommendation. Fix the three lying gates first (half a day, P0). Then move testing from "logic mirrored in tests" to "production entry points invoked by tests" and set a coverage floor (P1). Then harden CI reproducibility and tighten the compiler (P2, P3). A sequenced plan is in section 5.

1. Current gate scorecard

Gate	Runs locally	Runs in CI	Verdict today	Trustworthy?
Typecheck (<code>tsc --noEmit</code>)	check	yes	clean	Yes. Note it also typechecks generated <code>.next/dev/types</code> , which a crashed dev server can leave torn.
Lint (eslint, max-warnings 0)	check	yes	clean	Yes.
Unit tests (<code>bun test</code>)	check	yes	93 pass	Partly. 76.7% lines across loaded files only; routes, actions, hooks, components never loaded.
Production build	check	yes	success, ~5 s	Yes.
Dependency audit	not in check	yes, true	16 advisories, 2 critical	No. Cannot fail. Currently masking Next.js RCE advisories.
Smoke (dev server, 13 checks)	check	yes	13/13, or "exited early" if any dev server is open	Fragile locally; fine on a clean runner.
QA gate (5 phases × 10)	not in check	yes	phase 4: 8/10	Partly. Fails open when offline; P4-10 inherits the smoke fragility.
Bun / Node / Actions pinning	—	—	none	<code>bun-version: latest</code> , <code>macos-latest</code> , <code>tag-pinned actions</code> , no <code>packageManager</code> .

Component / DOM tests, e2e, accessibility	—	—	none	No DOM environment or testing library installed.
---	---	---	-------------	--

2. Findings and recommendations

Priority: P0 = gate currently lying or hiding a real risk · P1 = large blind spot · P2 = reproducibility and cost · P3 = strictness and polish. Effort is a working estimate for one engineer.

P0 Dependency audit cannot fail, and is hiding two critical Next.js advisories

Evidence. `.github/workflows/ci.yml:35` runs `bun audit --audit-level=high || true`. Running the audit today reports 16 advisories (2 critical, 14 high). The critical ones are in the direct dependency `next`, range `>=16.0.0 <16.3.3`, installed 16.2.12: unauthenticated RCE on Windows-hosted servers and RCE in the image optimisation API with AVIF input. `sharp <0.35.4` carries a high. Twelve of the sixteen live only in the eslint tool chain. The QA gate's P4-9 does fail on this, but its result was masked too because P4-10 failed for an unrelated reason (below), so the signal looked like noise.

Change.

- Upgrade `next` and `eslint-config-next` to 16.3.6, raise the `sharp` override to `^0.35.4`, run the full gate.
- Split the audit: `bun audit --prod --audit-level=high` as a hard-failing step (today it reports exactly the 4 real advisories: `nanoid`, `next` x2, `sharp`), and a second non-blocking `bun audit` for the dev tree whose output is uploaded as a job summary.
- Make the QA gate's `prodAdvisories()` fail closed: a registry error is a FAIL with the reason printed, not a PASS. Allow an explicit `--offline` flag that marks the check SKIPPED and exits non-zero unless `ALLOW_OFFLINE_QA=1`.
- Record dev-tree exceptions in `docs/security-exceptions.md` with an expiry date, as the July re-audit recommended (it was never written).

Effort: 2–3 h including the Next upgrade smoke run.

P0 Smoke and QA gates fail with no explanation whenever a dev server is open

Evidence. Next 16 keeps a per-project dev lock (`.next/dev/lock`); it is not per-port. With the Desktop launcher's server alive on port 3000, `node scripts/smoke.mjs` on port 3199 prints only `server exited early (code 1)`, and the QA gate's P4-10 reports `smoke:false`. The real message ("Another next dev server is already running ... PID ...") is captured into `serverLog` but never printed on this path (`scripts/smoke.mjs:51–64`). This is the exact failure that hid today's Rosetta outage.

Change.

- Print the last 20 lines of the captured server log on any early exit.
- Pre-flight: if `.next/dev/lock` exists and its PID is alive, fail immediately with "dev server PID n holds the lock; stop it or run with `--prod`".
- Make production mode the default in CI. The build has just run; `next start` boots in under a second and tests the artefact that ships rather than the dev server. Keep the dev-mode run as a second step so Turbopack dev regressions (the original reason for the script) are still caught.

- Add a `predev preflight ( scripts/preflight.mjs )` that fails fast when `process.arch` differs from `uname -m` or the `lightningcss` binary for the current arch is missing. It turns today's HTTP 500 on every page into a one-line error at start-up.

Effort: 2 h.

P0 Local `bun run check` is a lower bar than CI

Evidence. `package.json:18` chains `typecheck`, `lint`, `test`, `build`, `smoke`. CI additionally runs the dependency audit and all 50 QA checks. A contributor who sees local green has not run the same gates.

Change. Make `check` the full CI set and have CI call `bun run check` as one step, so the two can never drift. Keep a `check:fast` (`typecheck`, `lint`, `test`) for inner-loop use.

Effort: 30 min.

P1 Tests never invoke the production entry points

Evidence. `bun test --coverage` loads only 9 of the source files:

File	Funcs	Lines	Note
lib/cleanup-targets, process-identity, scoring, single-flight	100	100	
lib/probe.ts	90.9	98.6	
lib/schemas.ts	81.8	82.9	error branches untested
lib/guard.ts	60.0	59.4	lines 51–61 (the reject path) untested
lib/resolve-host.ts	20.0	29.8	
lib/sampler.ts	71.4	19.3	orchestration (lines 118–261) untested; only parsers are
lib/pool.ts, lib/format.ts, lib/utils.ts	not loaded		
app/api/{stats,scan,cleanup,privacy}/route.ts	not loaded		only black-box status codes via smoke; QA gate checks for the literal string <code>assertLocalRequest(</code> in source
app/actions.ts (<code>cleanupItem</code> , <code>killProcess</code>)	not loaded		<code>tests/cleanup-boundary.test.ts</code> re-implements <code>sweep()</code> instead of calling it
hooks/use-polling.ts, components/** (10 files)	not loaded		no DOM environment installed

Change.

- Route handler tests: import each `GET / POST` and call it with a constructed `Request`. Assert on the parsed body against `lib/schemas.ts`, the 403 path for a forged Host, and the `no-store` header. Bun runs these without a server.
- Server action tests: call `cleanupItem` and `killProcess` with a stubbed probe layer (inject via the existing `probe.ts` seam) and assert the refusal paths: path outside the permitted roots, PID 0/1, own PID, non-numeric input.

- Sampler orchestration: run `sample()` against fake probe results covering `timeout`, `denied`, `unsupported`, and confirm history window and alert state transitions. This lifts the 19% line figure and covers the code that actually runs every poll.
- Hook and component tests: add `happy-dom` and `@testing-library/react`, register via `bunfig.toml [test] preload`. First targets: `use-polling` (pause on hidden tab, overlap guard, cancellation) and `process-table` (sorting, kill confirmation). These replace the regex-on-source checks in QA phase 3 with behavioural ones.
- Coverage floor: `bunfig.toml [test] coverage = true, coverageThreshold = { lines = 0.80, functions = 0.85 }, coverageSkipTestFiles = true`. Ratchet upward as the above lands. Bun fails the run when the threshold is missed.

Effort: 2–3 days for the full set; the route and action tests (highest value) are about one day.

P1 No end-to-end or accessibility gate

Evidence. The smoke script checks status codes, two headers, and the presence of one heading. Accessibility is asserted only by QA phase 3 grepping source text for live regions and labels. The July research notes recommended a Playwright + axe gate; it was not built.

Change. One Playwright spec against the production server: page loads, stats populate within the polling interval, a process row can be selected, the cleanup confirmation appears and can be cancelled, and `axe-core` reports zero serious or critical violations. Run on the macOS runner after the build. Keep it to a single spec file so it stays fast and unflaky.

Effort: 1 day.

P2 CI is unpinned, uncached, unbounded, and runs twice per push

Evidence. `ci.yml`: `bun-version: latest`; `runs-on: macos-latest`; `actions/checkout@v4` and `oven-sh/setup-bun@v2` by tag; no `concurrency`; no `timeout-minutes`; no cache for `~/.bun/install/cache`; triggers on `push: ["**"]` and `pull_request`, so a push to a PR branch runs the job twice. No `packageManager` field, no `.node-version`, no Dependabot config.

Change.

- `"packageManager": "bun@1.3.5"` in `package.json` and `bun-version-file: package.json` in the workflow; `.node-version = 22` for the smoke and QA scripts that run under node.
- Pin actions to commit SHAs with a version comment; pin the runner to `macos-15`.
- `concurrency: { group: ci-${{ github.ref }}, cancel-in-progress: true }`; `timeout-minutes: 20`; restrict push triggers to `main` and let `pull_request` cover branches.
- Cache `~/.bun/install/cache` keyed on `hashFiles('bun.lock')`.
- `.github/dependabot.yml`: weekly, grouped minor/patch, separate security updates for `next`, `sharp`, `lightningcss`, and `github-actions`.
- Optional second job: `arch -x86_64` install and test on the arm64 runner, which would have caught today's Rosetta failure in CI rather than on the desktop.

Effort: half a day.

P3 Compiler strictness has cheap headroom

Evidence. `tsconfig.json` sets `strict` only. Measured against the current tree:

Flag	New errors	Where
<code>exactOptionalPropertyTypes</code>	0	free
<code>noImplicitOverride</code> + <code>noFallthroughCasesInSwitch</code>	0	free
<code>noUncheckedIndexedAccess</code>	38	17 in tests, 9 in page.tsx, 6 in sparkline.tsx, 6 in lib (sampler, resolve-host, pool, guard)
<code>noPropertyAccessFromIndexSignature</code>	197	not worth it

Change. Enable the three free flags now. Enable `noUncheckedIndexedAccess` in a dedicated PR: the 6 library hits are in code that parses `ps` and `vm_stat` output, which is where an unchecked index is most likely to be a real bug. Also make the safe Next defaults explicit so a regression is a visible diff: `typescript: { ignoreBuildErrors: false }, eslint: { ignoreDuringBuilds: false }, reactStrictMode: true, typedRoutes: true`. Exclude `.next/dev/types` from `tsc` runs, or clean it in `pretest`, so a torn generated file cannot fail typecheck.

Effort: 1 h for the free flags and config; half a day for the index-access PR.

P3 No local guardrails before commit

Evidence. No Prettier config, no lefthook or husky, no knip. Lint and format are enforced only in CI; unused exports and dependencies are never checked.

Change. `lefthook.yml` running `tsc --noEmit` and `eslint` on staged files at pre-commit; Prettier with the default config and a CI `--check`; `knip` in CI to catch unused files, exports, and dependencies (the removed `shadcn` import that once caused HTTP 500 on every page is precisely the class `knip` catches).

Effort: 2 h.

3. Target pipeline

Stage	Contents	Time budget
Pre-commit (lefthook)	<code>tsc</code> , <code>eslint</code> on staged, <code>prettier</code>	< 10 s
<code>bun run check:fast</code>	<code>typecheck</code> · <code>lint</code> · unit tests with coverage floor	< 30 s
<code>bun run check</code> (= CI)	<code>check:fast</code> · prod audit (hard fail) · <code>build</code> · <code>smoke --prod</code> · <code>smoke dev</code> · QA gate all · <code>e2e + axe</code> · <code>knip</code>	< 6 min on macos-15
Nightly / weekly	full-tree audit report · Dependabot PRs · optional x86_64 leg	—

4. Why not the alternatives

- **Re-adding the x64 lightningcss binary** would have made today's launcher work, but it was reverted as M-13 for good reason: it is a platform-locked dependency that masks an environment fault. The

preflight check gives the same protection with a clear message instead.

- **Raising `--audit-level` to `critical`** to make the step pass is the wrong direction; the two criticals are in `next` itself. Upgrade and fail closed.
- **Replacing the QA gate with conventional tools** is not recommended. Its 50 checks encode project-specific invariants (loopback-only, permitted deletion roots, typed probe outcomes) that generic tooling cannot express. The fix is to have it call real code rather than `grep` source text, and to fail closed.

5. Sequenced plan

Step	Work	Exit criterion
1 (P0, ½ day)	Next 16.3.6 + sharp override; audit split and hard fail; QA gate fails closed offline; smoke prints server log, checks dev lock, prod mode in CI; <code>check</code> = CI; preflight arch check on <code>predev</code>	CI red on any prod advisory; QA phase 4 10/10; smoke gives a one-line reason on any failure
2 (P2, ½ day)	<code>packageManager</code> , SHA-pinned actions, macos-15, concurrency, timeouts, bun cache, Dependabot	Same lockfile and tool versions on every run; one CI run per push
3 (P1, ~1 day)	Route handler and server action tests; sampler orchestration tests; coverage floor 80/85	Every <code>src/app</code> entry point imported by a test; sampler above 80% lines
4 (P1, ~2 days)	happy-dom + testing-library; use-polling and process-table tests; Playwright + axe spec	QA phase 3 regex checks replaced by behavioural tests; zero serious axe violations
5 (P3, ½ day)	Free <code>tsconfig</code> flags, explicit Next config, lefthook, Prettier, knip; then <code>noUncheckedIndexedAccess</code> PR	Zero new lint or type debt; unused exports fail CI

Appendix A. Measurements taken

```
bun run typecheck          clean (after the healthy dev run regenerated
.next/dev/types/validator.ts)
bun run lint                clean
bun test --coverage        93 pass · 80.46% funcs · 76.66% lines (loaded files only)
bun run build              success in 5.3 s (native arm64, node 22.17.0)
node scripts/smoke.mjs     13/13 when no other dev server is running; "exited early (code
1)" otherwise
node scripts/smoke.mjs --prod 13/13
node scripts/qa-gate.mjs all phases 0–3: 10/10 · phase 4: 8/10 (P4–9 advisories, P4–10
smoke)
bun audit --audit-level=high 16 advisories (2 critical, 14 high), exit 1; CI step ignores
exit code
bun audit --prod           4 advisories: nanoid <3.3.18, next >=16.0.0 <16.3.3 (×2
critical), sharp <0.35.4
npm view next version      16.3.6 (installed 16.2.12)
tsc --noUncheckedIndexedAccess 38 errors · --exactOptionalPropertyTypes 0 · --
noImplicitOverride --noFallthroughCasesInSwitch 0
```

Appendix B. Sources

- `.github/workflows/ci.yml`, `package.json`, `tsconfig.json`, `next.config.ts`, `scripts/smoke.mjs`, `scripts/qa-gate.mjs`, `tests/*.test.ts`
- `DEEP_APP_REAUDIT_2026-07-29.md` findings HIGH-04 (false-green gates) and MEDIUM-04 (coverage), both re-verified open
- `docs/research.md` §1 (SHA pinning), §8 (Playwright + axe), both recommended in July and not yet implemented
- Today's localhost outage: Desktop launcher under Rosetta, documented in `README.md` "Troubleshooting" on this branch